

Allocation Patterns

- Map-Reduce
- **Multi-Tier**

Multi-Tier Pattern

- **Context:** In a distributed deployment, there is often a need to distribute a system's infrastructure into distinct subsets.
 - This may be for operational or business reasons (for example, different parts of the infrastructure may belong to different organizations).
- **Problem:** How can we split the system into a number of computationally independent execution structures—groups of software and hardware—connected by some communications media?
- **Solution:** The execution structures of many systems are organized as a set of logical groupings of components. Each grouping is termed a tier.
 - The grouping of components into tiers may be based on a variety of criteria, such as the type of component, sharing the same execution environment, or having the same runtime purpose.

Multi-Tier Example

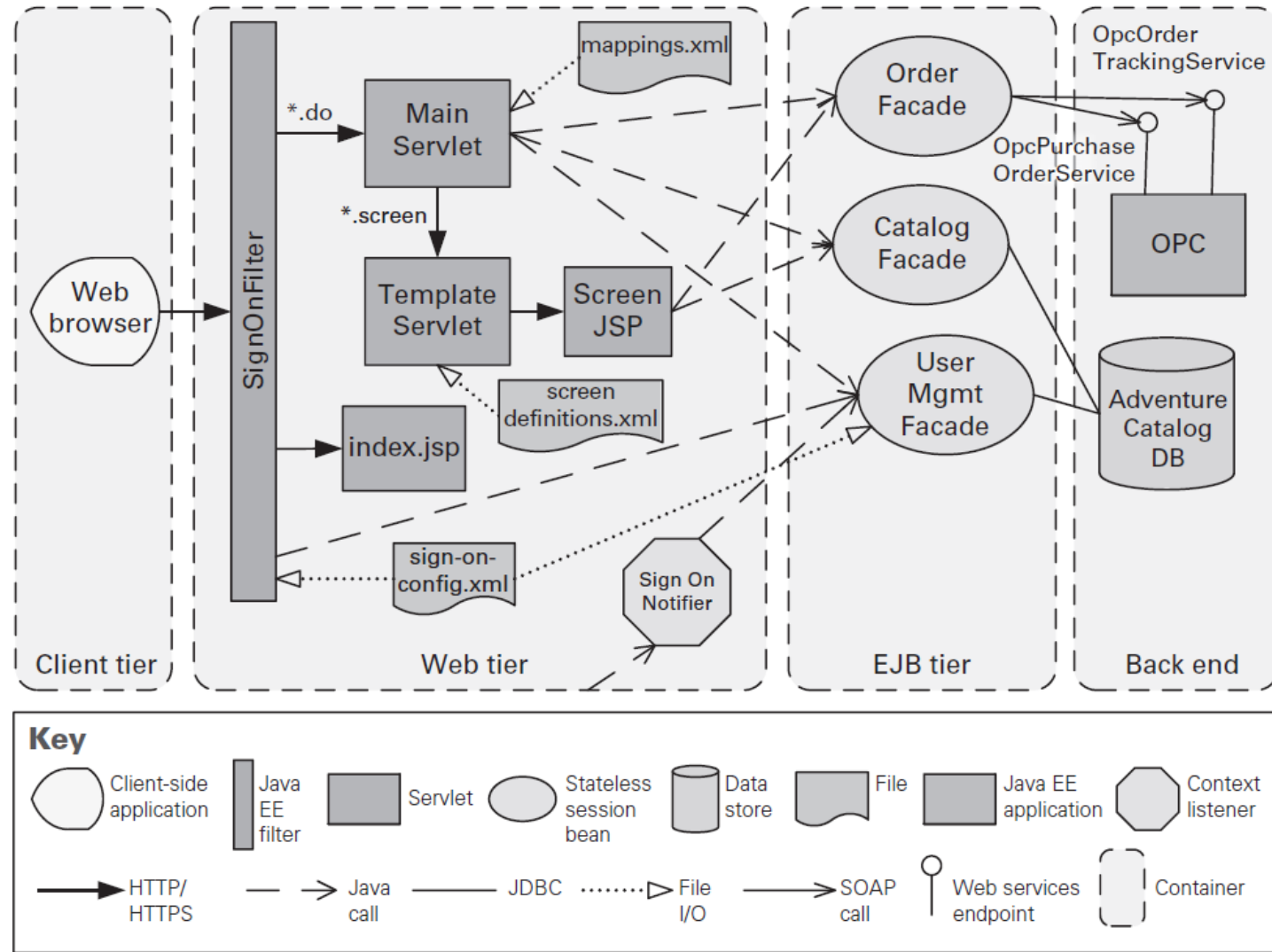


FIGURE 13.15 A multi-tier view of the Consumer Website Java EE application, which is part of the Adventure Builder system

Multi-Tier Solution-1

- Overview: The **execution structures** of many systems are **organized** as a set of logical **groupings of** components. Each grouping is termed a *tier*.
- **Elements:**
 - *Tier*, which is a logical grouping of software components.
- **Relations:**
 - *Is part of*, to **group components into tiers**.
 - *Communicates with*, to **show how tiers and the components they contain interact with each other**.
 - *Allocated to*, in the case that **tiers map to computing platforms**.

Multi-Tier Solution-2

- **Constraints:**

- A software component belongs to exactly one tier.
- Tiers induce topological constraints that restrict which components may communicate with other components. Specifically, connectors may exist only between components in the same tier or residing in adjacent tiers.
- Tiers may constrain the kinds of communication that can take place across adjacent tiers. For example, some tiered patterns require call-return communication in one direction but event-based notification in the other.

Multi-Tier Solution-3

- **Weaknesses:**

- Substantial up-front cost and complexity. For simple systems, the benefits of the multi-tier architecture may not justify its up-front and ongoing costs, in terms of hardware, software, and design and implementation complexity.

- **Advantages:**

- Tiers make it easier to ensure security, and to optimize performance and availability in specialized ways.
- They also enhance the modifiability of the system, as the computationally independent subgroups need to agree on protocols for interaction, thus reducing their coupling.

Finer Points of Multi-Tier

- Tiers are not components, but rather logical groupings of components.
- Don't confuse tiers with layers!
 - Layering is a pattern of modules (a unit of implementation),
 - while tiers applies only to runtime entities.